Google Workspace Day-2 Administration & Operations Runbook (OU Design, Policy Governance & Runbooks)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Solution Overview 4

What this is . . . 4

How it works . . . 5

Reference operating model . . . 6

Deployment model . . . 6

Benefits . . . 7

Limitations and boundaries . . . 7

Requirements 7

R1. A completed go-live (steady-state, not mid-migration) . . . 8

R2. Super-admin plus a defined admin-team structure . . . 8

R3. Reports API / audit-log access and retention posture . . . 8

R4. A control-plane identity for the Python tooling (OAuth or domain-wide delegated service account) 9

R5. A documented target-state design (or the intent to produce one) . . . 9

R6. Change-control and evidence process (extends GR-5) . . . 9

Global Requirements — Google Workspace & Gemini Solutions 10

1. Google Workspace / Cloud Identity tenant . . . 10

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 10

3. DNS control for the routed domains . . . 11

4. Identity & provisioning source of truth . . . 11

5. A pilot scope (OU / group) and change control . . . 11

6. Security, compliance & governance stakeholders . . . 11

Implementation 12

Stage 1 — Assess (read-only baseline) . . . 12

Stage 2 — Design . . . 13

Stage 3 — Configure (pilot scope first) . . . 15

Stage 4 — Pilot . . . 16

Stage 5 — Validate (pass/fail gates) . . . 16

Stage 6 — Expand . . . 17

Testing & Validation 17

T1 — OU tree matches design (supports Gate G1) . . . 17

T2 — Settings baseline applied, zero drift (Gate G1) . . . 18

T3 — Inheritance behaves as designed . . . 18

T4 — Least-privilege delegation (Gate G2) . . . 18

T5 — Alerts fire on test events (Gate G3) . . . 19

T6 — Health-check report generates from real data (Gate G3) . . . 19
T7 — Runbooks executed by delegated role (Gate G4) . . . 19
T8 — Drift detection catches an unplanned change . . . 20
Validation summary . . . 20
Troubleshooting 20
Authentication and API access (the Python tools) . . . 20
OU tree and settings . . . 21
Admin roles and delegation . . . 22
Monitoring and alerts . . . 22
Runbooks and lifecycle helper . . . 23
Compliance boundary reminder . . . 23
Best Practices 23
OU tree and policy governance . . . 23
Least-privilege administration . . . 24
Monitoring, alerting, and health . . . 24
Runbooks and change control . . . 24
Tooling and credentials . . . 25
Compliance posture . . . 25
Deliverables Summary 25
Documentation . . . 25
The 5 Python tools . . . 26
Appendix A — PowerShell Scripts 27

Executive Summary

The outcome. When a Google Workspace project goes live, the organization needs to *run* it — not just have it. This solution delivers the **day-2 administration and operations model**: a governed organizational-unit (OU) design and configuration-policy baseline, a least-privilege admin-role delegation model, live monitoring and health checks, and a library of runbooks for every routine and incident task. The result is operations that are **owned, repeatable, and auditable** from the first day of business-as-usual, instead of go-live firefighting and tribal knowledge.

The approach. It follows a disciplined, pilot-first model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — with explicit pass/fail gates and a documented rollback for every production change. Assess the current OU/policy/role/alert state (read-only); design the target OU tree, settings baseline, delegation matrix, alert set, and runbook library; configure and prove it on a single pilot OU and help-desk team; validate against four gates (OU/policy applied with zero drift, least-privilege confirmed, monitoring live, runbooks exercised without super-admin); then expand OU-by-OU and hand to the BAU team. Five read-only/dry-run **Python tools** (Google Workspace Admin SDK — Directory and Reports APIs) automate assessment, config-drift detection, admin-role auditing, health reporting, and safe operations helpers — with no credentials or keys in code.

Why it is low-risk. Nothing rolls to the whole organization until the pilot passes every gate. Tooling is read-only or dry-run by default, so assessment and validation never change the tenant. Every production change (policy, roles, sharing, mail-related settings) carries a rollback, and expansion is batched OU-by-OU so any batch can be reverted independently. Least-privilege delegation means day-to-day operations never require super-admin, and configuration is continuously checked against a documented baseline — so the tenant stays in its intended, auditable state. Vendor detail is drawn from Google's public documentation, with preview- or edition-dependent behavior flagged to verify in your tenant; regulatory *determinations* are deferred to your compliance and assessment partners, while this solution supplies the technical controls, monitoring, and evidence.

Delivered as exact, self-guided steps plus tested tooling — the same operating model a consulting engagement would install, at a fraction of the cost.

Solution Overview

What this is

The day a Google Workspace project goes live, the work does not end — it changes shape. Someone has to **run** the tenant: keep policies coherent as the organization grows, add and remove users, handle requests and incidents, watch for drift, and prove the whole thing is under control when audit season arrives. Without a structured operating model, "day-2" degrades into firefighting: policies applied inconsistently, admin rights handed out too broadly, no monitoring, and tribal knowledge that walks out the door when a key admin leaves.

This solution is the **Day-2 Administration & Operations Runbook** for Google Workspace. It delivers the **steady-state operating model** for the tenant after go-live, and it folds in the two structural foundations that make steady-state operations possible:

1. **Organizational-unit (OU) design and configuration-policy governance** — the OU tree, how

settings inherit down it, the documented settings baseline, and the change-control and config-drift discipline that keep the tenant's configuration matching its intended design.

1. **The day-2 operations runbook library** — the admin-role delegation model (least-privilege), monitoring/alerting and health checks, and a library of runbooks: routine SOPs, user/group/device lifecycle tasks, request handling, incident handling, and escalation.

The result is an operations practice that is **owned, repeatable, and auditable** from go-live onward, rather than reinvented on every ticket.

How it is distinct from the adjacent solutions

• It is **not** the adoption/change-management and end-user training program (that is a separate adoption solution) — this is what the *administrators* do to keep the service running.

• It is **not** the migration cutover operations solution (mailbox/Drive/Chat data movement, cutover weekend runbook) — this is what happens **after** cutover, in business-as-usual (BAU) steady state.

If you have already migrated to Google Workspace and now need to *operate* it well, this is that solution.

How it works

The operating model rests on four pillars, each with a corresponding section of the implementation guide and at least one Python tool:

1. OU tree and policy governance

Google Workspace applies almost all service configuration (Gmail, Drive, Chat, Meet, Calendar, security settings, Marketplace app allow-lists, and per-service on/off state) through **settings that attach to an organizational unit** and **inherit** from parent OU to child OU unless overridden. A deliberate OU tree is therefore the backbone of governance: it decides which populations get which policy, and where exceptions live. This solution designs the OU tree, documents the intended **settings baseline** per OU, and establishes the discipline to keep the live configuration matching that baseline (change control + drift detection). Configuration groups (access groups used for settings targeting) are covered where a policy must follow membership rather than OU placement — verify group-based policy availability for the specific service in your tenant.

2. Admin-role delegation (least-privilege)

Super-admin is all-powerful and must be rare. Google Workspace ships **pre-built admin roles** (Groups Admin, User Management Admin, Help Desk Admin, Services Admin, and others) and supports **custom admin roles** scoped to specific privileges and, optionally, to specific OUs. This solution defines a least-privilege delegation model: which team gets which role, scoped to which OU, so that a help-desk agent can reset passwords but cannot alter security policy, and day-to-day work never requires super-admin.

3. Monitoring, alerting, and health checks

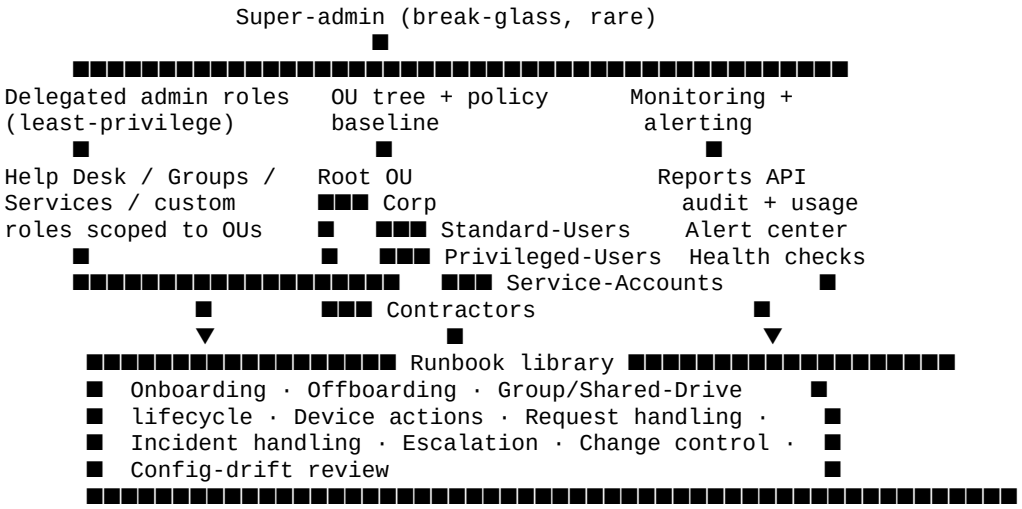
Google Workspace exposes operational signal through the **Admin console**, the **Reports API / audit and usage logs** (login, admin, drive, token, and other activity events), and the **alert center** (admin.google.com → Security → Alert center) with **system-defined and admin-created alerts**. This solution stands up a repeatable **health-check** routine and a monitoring/alerting baseline so that security-relevant events

(suspicious logins, admin privilege changes, external sharing spikes) and service-health issues surface proactively rather than being discovered by a user complaint.

4. The runbook library

Everything routine becomes a named, repeatable runbook: onboarding/offboarding, group and Shared Drive lifecycle, device actions, common request handling, incident handling (account compromise, data exposure, mail-flow issue), and the escalation path. Runbooks are the artifact that makes operations survivable when people change roles.

Reference operating model



The OU tree above is an illustrative reference; the real tree is designed to your populations in the implementation guide. Naming convention for every artifact this solution creates is `<Purpose> – <Scope> – <Stage>`, for example `OU – Corp/Privileged-Users – Baseline`, `Admin Role – Help Desk – Corp`, `Alert – External-Sharing-Spike – Enforce`, `Runbook – User Offboarding – Production`.

Deployment model

This solution follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-first, with explicit pass/fail gates and a documented rollback for every production change:

• **Assess** — inventory the current OU tree, per-OU settings, admin-role assignments, alert configuration, and existing (often undocumented) operational practices. The read-only assessment tool produces this baseline.

• **Design** — the target OU tree, the settings baseline per OU, the admin-role delegation matrix, the monitoring/alert set, and the runbook library table of contents.

• **Configure** — build the OU tree, apply the settings baseline, create the delegated admin roles, configure alerts, and author the runbooks.

• **Pilot** — apply to one **pilot OU** (`OU – Corp/Standard-Users – Pilot`) and one help-desk team; exercise the onboarding, offboarding, request, and incident runbooks against pilot users only.

• **Validate** — confirm the pilot passes the gates (settings match baseline, drift detector clean, alerts fire on test events, runbooks executed successfully with no super-admin required).

• **Expand** — roll the OU/policy structure, roles, alerts, and runbooks to the rest of the organization, then hand to the operations team as owned BAU.

Benefits

• **No firefighting.** Every routine task has a named runbook; every population has a defined policy.

• **Least-privilege by design.** Day-to-day admin work never touches super-admin.

• **Auditable.** Configuration matches a documented baseline; drift is detected; admin and access events are monitored and retained via the Reports API.

• **Survivable.** Operations knowledge lives in runbooks, not in one person's head.

• **Self-guided.** Delivered as exact steps plus read-only/dry-run Python tooling, at a fraction of the cost of a consulting engagement.

Limitations and boundaries

• The **Google Admin console and Cloud Identity** steps are authored from Google's public documentation. Where a capability is in preview or is rolling out (some group-based policy targeting, newer alert types, certain Gemini/AI admin controls), it is flagged **"verify in your tenant."**

• **Regulatory and compliance interpretation** — whether a control satisfies CMMC/NIST 800-171, ITAR/DFARS applicability, or an assessor's determination — is **deferred to your compliance, legal, and assessment (C3PAO) partners.** This solution provides the technical control configuration, monitoring, and evidence; it does not make certification determinations.

• The Python tools automate **assessment, validation, health reporting, and drift detection**; they are **read-only or dry-run by default** and never contain credentials. They complement, not replace, the documented Admin console procedures.

• Exact **audit-log field names, alert types, and admin-role privileges** evolve; the solution names the current documented ones and directs you to verify against your tenant's current console.

Requirements

These are the **solution-specific** prerequisites for standing up a Google Workspace day-2 administration and operations practice. They build **on top of** the shared Google-category prerequisites in `global-requirements-google.md` — the provisioned Workspace/Cloud Identity tenant with super-admin (GR-1), a pilot OU/group and change control (GR-5), and engagement with security, compliance, and governance stakeholders (GR-6). Verify those first; the items below are what this operations solution additionally needs. Each is stated as **Description / Why required / How to verify / How to implement if missing**.

R1. A completed go-live (steady-state, not mid-migration)

- **Description:** Google Workspace is already in production — users are on Gmail/Drive/Docs/Meet/Chat

and the cutover is complete. This solution operates a live tenant, it does not migrate one.

- **Why required:** The operating model (OU baseline, drift detection, runbooks) assumes a stable

target configuration to govern *against*. Running it mid-migration produces false drift and unstable baselines.

- **How to verify:** Confirm mail flow (MX) points at Google, users sign in daily, and there is no

active migration wave in flight.

- **How to implement if missing:** Complete migration cutover first (that is the separate

migration-cutover-operations solution). Return here once BAU begins.

R2. Super-admin plus a defined admin-team structure

- **Description:** At least one **super-admin** account (used only for break-glass and for building

the delegation model), **plus** the list of teams/individuals who will operate the tenant day-to-day and what each must be able to do (help desk, identity/groups, service configuration, security).

- **Why required:** The least-privilege delegation model (`Admin Role – <team> – <scope>`) is built

from this. Without a defined team-to-capability mapping, roles cannot be scoped.

- **How to verify:** In **admin.google.com** → **Account** → **Admin roles**, confirm current super-admins

(there should be a small, known number) and current role assignments. Confirm the operations org chart / responsibilities off-console.

- **How to implement if missing:** Reduce super-admins to the minimum, establish break-glass

accounts, and document the operations team responsibilities before designing roles.

R3. Reports API / audit-log access and retention posture

- **Description:** Access to the **Admin SDK Reports API** (activity and usage reports) and a decision

on **audit-log retention** appropriate to your edition and compliance obligations. Google Workspace retains audit-log data for defined windows by service; longer retention typically requires export (e.g. to BigQuery) — verify the current retention windows and export options for your edition.

- **Why required:** Monitoring, health checks, and evidence (R6) read from the Reports API; incident

investigation depends on logs still being retained when you look.

- **How to verify:** In the Admin console, confirm the **Reporting / Audit and investigation** tools

are available for your edition; confirm the Reports API is enabled in the associated Google Cloud project (see R4).

- **How to implement if missing:** Enable the Reports API in the service-account/OAuth project;

if long-term retention is required, stand up a log export (e.g. BigQuery export) — verify the current export mechanism and windows for your edition.

R4. A control-plane identity for the Python tooling (OAuth or domain-wide delegated service account)

• **Description:** An authentication method for the five Python tools: either an **OAuth user credential **for an admin, or a** service account with domain-wide delegation** authorized for the read-only Admin SDK scopes the tools use (Directory read-only, Reports read-only, Groups settings read-only). No keys are ever stored in the tools.

• **Why required:** The tools call the Admin SDK Directory and Reports APIs to assess, validate, and detect drift. They must authenticate with least-privilege, credential-free code.

• **How to verify:** Confirm a Google Cloud project exists with the **Admin SDK API** enabled and either an OAuth client or a domain-wide-delegated service account authorized for the read-only scopes (in **Admin console** → **Security** → **API controls** → **Domain-wide delegation** the client ID is listed with only the read-only scopes).

• **How to implement if missing:** Create the Cloud project, enable the Admin SDK API, create the service account (or OAuth client), and authorize **only the read-only scopes** the tools name in their `--help`. Supply the credential to the tools via ADC / an environment path — never hardcoded.

R5. A documented target-state design (or the intent to produce one)

• **Description:** The **target OU tree**, the **per-OU settings baseline**, the **admin-role delegation matrix**, the** monitored events/alerts list**, and the** runbook library table of contents **— either already documented, or acknowledged as the first deliverable of the Design stage. The drift detector compares live configuration against a** baseline file** you own.

• **Why required:** "Drift" and "correct policy" are only meaningful relative to an intended design. The config-drift tool needs a baseline artifact to compare against.

• **How to verify:** Confirm a baseline document/JSON exists (or is scheduled) that names each OU and its intended key settings.

• **How to implement if missing:** Produce it in the Design stage (Implementation §Design) — the assessment tool exports the *current* state as a starting point you curate into the intended baseline.

R6. Change-control and evidence process (extends GR-5)

• **Description:** A **change-control** path for tenant configuration changes (who approves, the change window, the rollback plan) and a place to **retain evidence** (health-check reports, drift reports, role-audit exports) for audit.

• **Why required:** Governance requires that configuration changes are approved and reversible and

that the operating state is provable after the fact. Config-drift detection is only useful if unplanned changes are actually caught by a process.

• **How to verify:** Confirm an approval path and change window exist (GR-5) and that generated reports are stored in a durable, access-controlled location.

• **How to implement if missing:** Define the approval/rollback process with the tenant owners and designate an evidence store before making production configuration changes.

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the

required role.

- **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

- **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.
- **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.
- **How to verify:** Confirm access to the authoritative DNS zone for each domain.
- **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

- **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).
- **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.
- **How to verify:** Confirm which directory is authoritative and how objects flow into Google.
- **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

- **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).
- **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.
- **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.
- **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before making production changes.

6. Security, compliance & governance stakeholders

- **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance**

stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

Implementation

This guide builds the Google Workspace day-2 operations practice in the house order **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**. Work pilot-first: nothing rolls to the whole organization until the pilot passes the gates in §Validate. Every artifact you create is named with the convention `<Purpose> – <Scope> – <Stage>` and that exact name is reused in testing, troubleshooting, and the scripts. Production changes (policy, roles, mail-related settings) carry a documented rollback.

> ***Navigation note.*** *Admin console paths below (admin.google.com) reflect Google's public documentation at authoring time. Google occasionally relabels or relocates console sections and adds preview features. Where a path or capability may differ by edition or rollout, the step says **verify in your tenant**. Never assume; confirm against your current console.*

Stage 1 — Assess (read-only baseline)

Establish the current state before changing anything. This is entirely read-only.

1. **Run the discovery/assessment tool.** From a workstation with the R4 credential configured, run:

```
python ou_policy_assessment.py --output ou-policy-baseline-current.json
```

It enumerates the current **OU tree** (Directory API `orgunits.list`), the **users/admins per OU**, the **groups**, and a summary of **admin-role assignments**, and writes them to a JSON snapshot. It is read-only; it makes no changes.

1. **Run the admin-role/delegation audit.**

```
python admin_role_auditor.py --output admin-role-audit-current.csv
```

It lists every admin-role assignment (`roleAssignments.list`), flags **super-admins**, and flags roles assigned at the root vs. scoped to an OU. Save the CSV as the assessment artifact `Report – Admin-Role Audit – Assess`.

1. **Capture current monitoring/alert posture.** In **admin.google.com → Security → Alert center →

Settings**, note which system-defined alerts are enabled and which admin-created alert rules exist. Record whether the** Reports API** is enabled (R3/R4).

1. **Record undocumented practice.** Interview the current operators: how are users onboarded/

offboarded today, who resets passwords, how are incidents handled. This becomes the raw material for the runbook library.

Assess output: `ou-policy-baseline-current.json`, `Report – Admin-Role Audit – Assess`, an alert inventory, and notes on current practice.

Stage 2 — Design

Turn the assessment into an intended target design. These are documents/artifacts, not console changes yet.

2.1 Design the OU tree

1. Design a **shallow, purpose-driven OU tree** — populations that need different policy, not a mirror of the HR org chart. A common reference shape:

```
Root
■■■ Corp
■   ■■■ Standard-Users
■   ■■■ Privileged-Users        (admins, finance, legal — stricter policy)
■   ■■■ Service-Accounts        (no interactive policy; API/service identities)
■■■ Contractors                 (restricted sharing, no data export)
```

1. Name each OU artifact `OU – <path> – Baseline`, e.g. `OU – Corp/Privileged-Users – Baseline`.

2. Decide **inheritance intent** for each: which settings inherit from parent and which are overridden at the child. Prefer inheritance; override only where a population genuinely needs different policy.

1. Where a policy must follow **membership** rather than OU placement, note it for a **configuration group **(access-group-based policy targeting) instead of an OU override —** verify group-based policy availability for that specific service in your tenant.**

2.2 Design the settings baseline

For each OU, document the intended value of the **governance-critical settings** — the ones that carry security/compliance weight and are the highest-value drift targets:

- **Per-service on/off** (Gmail, Drive, Meet, Chat, Marketplace apps allow-listing).
- **Drive external sharing** (off / allow-listed domains / on) and **link-sharing defaults**.
- **2-Step Verification enforcement** and **login challenges**.
- **Less-secure/legacy access, session length, and context-aware access** where in scope.
- **Gmail routing/compliance/attachment settings** where governed.

Record these in a **baseline file** — `ou-policy-baseline.json` — that the drift detector will compare live configuration against. Curate it from the Assess snapshot (edit the current-state export into the intended state). Name it `Baseline – OU Settings – Design`.

2.3 Design the admin-role delegation matrix

Map each operations team to the **least-privilege** role and scope:

Team	Role artifact	Base role	Scope
Tier-1 help desk	`Admin Role – Help Desk – Corp`	Help Desk Admin (custom, reset password + read user)	`OU – Corp`
Identity/groups	`Admin Role – Groups – Corp`	Groups Admin	`OU – Corp`
Service config	`Admin Role – Services – Root`	Services Admin (scoped services only)	Root or per-service
Security ops	`Admin Role – Security-Read – Root`	Custom (security/report read, alert center)	Root
Break-glass	(super-admin, kept minimal)	Super Admin	Root

Custom roles let you grant a **specific privilege set** and, for user-management privileges, **scope them to an OU** — verify which privileges are OU-scopable in your tenant. The design principle: no day-to-day task requires super-admin.

2.4 Design monitoring/alerting and health checks

1. Define the **monitored events** — the security- and operations-relevant ones the health-check tool

will report from the **Reports API** (login failures/suspicious logins, admin activity, external sharing, OAuth token grants, mobile/device events).

1. Define the **alerts** to enable/create in the alert center, each named `Alert – <condition> –

Enforce`, `e.g.` Alert – Suspicious-Login – Enforce`,` Alert – Admin-Privilege-Change – Enforce`,` Alert – External-Sharing-Spike – Enforce`. Use system-defined alerts where they exist; create admin rules for the rest — **verify available alert types in your tenant.**

1. Define the **health-check cadence** (e.g. daily automated report, weekly review) the report

generator produces.

2.5 Design the runbook library (table of contents)

List every runbook, each named `Runbook – <task> – Production`:

- `Runbook – User Onboarding – Production`
- `Runbook – User Offboarding – Production`
- `Runbook – Group Lifecycle – Production`
- `Runbook – Shared Drive Lifecycle – Production`
- `Runbook – Device Action – Production` (wipe/block/approve)
- `Runbook – Access Request Handling – Production`
- `Runbook – Incident: Account Compromise – Production`
- `Runbook – Incident: Data Exposure / Oversharing – Production`

- `Runbook – Incident: Mail-Flow Issue – Production`
- `Runbook – Change Control – Production`
- `Runbook – Config-Drift Review – Production`

Design gate: the OU tree, baseline file, role matrix, alert list, and runbook TOC are documented and approved by the change-control and security stakeholders (GR-5/GR-6) before any Configure step.

Stage 3 — Configure (pilot scope first)

Build the design. Apply every change to the **pilot OU** (`OU – Corp/Standard-Users – Pilot`) and the pilot help-desk team **before** the rest of the organization. Each production change below has a rollback.

3.1 Build the OU tree

1. In **admin.google.com** → **Directory** → **Organizational units**, create the OUs from the Design tree.

Create parents before children.

1. Move a **small pilot set of test/volunteer users** into `OU – Corp/Standard-Users – Pilot`.

Leave everyone else where they are.

- **Rollback:** OUs can be deleted only when empty; moving users back to their prior OU reverts

inherited policy. Record each user's original OU before moving.

3.2 Apply the settings baseline to the pilot OU

1. For each governed service, set the intended value from `Baseline – OU Settings – Design` **on the

pilot OU only **(e.g.** Apps → Google Workspace → Drive and Docs → Sharing settings**, scoped to the pilot OU;** Security → Authentication → 2-Step Verification**, scoped to the pilot OU). Leave parent/root values unchanged so inheritance elsewhere is untouched.

1. After applying, run the drift detector against the pilot OU to confirm live == baseline:

```
python config_drift_detector.py --baseline ou-policy-baseline.json --scope Corp/Standard-Users/Pilot
```

Expect **0 drift** for the pilot OU.

- **Rollback:** set the pilot OU settings back to **Inherit from parent**, which restores the

pre-change behavior.

3.3 Create the delegated admin roles

1. In **Account** → **Admin roles**, create each **custom role** from the delegation matrix with only its

intended privileges; assign it to the **pilot help-desk member**, scoped to the pilot OU where supported.

1. Verify with the auditor:

```
python admin_role_auditor.py --output admin-role-audit-pilot.csv
```

Confirm the pilot help-desk member has exactly the intended role and **no super-admin**.

- **Rollback:** remove the role assignment; the account reverts to no admin privileges.

3.4 Configure monitoring, alerting, and health checks

1. In **Security** → **Alert center** → **Settings**, enable the system-defined alerts from the Design list and create the admin alert rules (`Alert – … – Enforce`). Set notification recipients to the security-ops group. **Verify available alert types in your tenant.**

1. Confirm the Reports API is enabled and the R4 credential can read it. Generate the first health report:

```
python health_check_report.py --days 7 --output health-check-pilot.html
```

• **Rollback:** disabling an admin-created alert rule reverts to the prior (no-alert) state; the health report is read-only and needs no rollback.

3.5 Author the runbook library

1. Write each `Runbook – <task> – Production` from the Design TOC. Every runbook states: **trigger, pre-checks, exact Admin console / Admin SDK steps, the artifact names touched, verification, and escalation.** Use the ops/config helper to standardize inputs and dry-run destructive lifecycle steps:

```
python ops_config_helper.py offboard --user pilot.user@example.com --dry-run
```

(`--dry-run` is the default; it prints the offboarding checklist and the API calls it *would* make without changing anything.)

1. Store runbooks in the operations knowledge base alongside the evidence store (R6).

Stage 4 — Pilot

Operate the pilot as if it were production, for a defined window (e.g. one to two weeks):

• Run `Runbook – User Onboarding – Production` and `Runbook – User Offboarding – Production` against pilot users, using the pilot help-desk role (not super-admin).

• Fire a **test event** for each configured alert (e.g. a controlled external-share of a pilot document) and confirm the alert appears in the alert center and notifies the recipients.

• Run the daily `health_check_report.py` and the weekly `config_drift_detector.py` for the pilot OU.

Stage 5 — Validate (pass/fail gates)

Do not expand until **all** gates pass. See `05-testing.md` for the full procedures.

• **Gate G1 — OU/policy applied:** `config_drift_detector.py --scope Corp/Standard-Users/Pilot` reports **0 drift** against `ou-policy-baseline.json`. **Fail** → reconcile the setting or the baseline and re-run.

• **Gate G2 — Least-privilege confirmed:** `admin_role_auditor.py` shows the pilot help-desk member

with **only** the intended role and **no unexpected super-admins**. **Fail** → remove excess privilege.

• **Gate G3 — Monitoring live:** every `Alert – … – Enforce` fired on its test event and notified the

security-ops group; the health report generated with real Reports API data. **Fail** → fix the alert rule / API access.

• **Gate G4 — Runbooks exercised:** onboarding, offboarding, a request, and one incident runbook were

executed successfully by the delegated (non-super-admin) role, with correct verification. **Fail** → fix the runbook or the role scope.

Stage 6 — Expand

1. Roll the OU tree and settings baseline to the remaining populations **OU by OU**, running

`config_drift_detector.py` after each to confirm 0 drift. Move users into their target OUs in controlled batches (record original OU for rollback).

1. Assign the delegated admin roles to the full operations teams; re-run `admin_role_auditor.py` and

confirm the super-admin count is still minimal.

1. Enable the full alert set organization-wide; schedule the health report (daily) and the drift

detector (weekly) as recurring jobs against the R4 read-only credential.

1. Publish the full runbook library and hand operations to the BAU team as **owned** — every routine

task has a named runbook, every population a governed policy, and monitoring is live.

Expansion rollback: because expansion is OU-by-OU, any batch can be reverted by moving its users back to their original OU and setting the OU's settings to **Inherit**, restoring pre-change behavior for that population without affecting others.

Testing & Validation

These procedures prove the day-2 operations practice works before it is handed to BAU. They map one-to-one to the pass/fail gates in `03-implementation.md` §Validate. Run every test against the **pilot scope** first; re-run the relevant tests after each Expand batch. Each test states its procedure and the **expected pass** vs. **fail** signal.

> *Convention: tests reference the exact artifact names created during implementation (`OU – Corp/Standard-Users – Pilot`, `Baseline – OU Settings – Design`, `Admin Role – Help Desk – Corp`, `Alert – … – Enforce`, `Runbook – … – Production`).*

T1 — OU tree matches design (supports Gate G1)

- **Procedure:** Run `python ou_policy_assessment.py --output ou-tree-check.json` and compare the

enumerated OU tree against the Design tree. Confirm each `OU – <path> – Baseline` exists with the intended parent.

- **Expected PASS:** every designed OU is present with the correct parent; the pilot users are in

`OU – Corp/Standard-Users – Pilot` and no one else was moved.

- **FAIL:** a designed OU is missing, mis-parented, or extra users were moved. → Correct the OU

structure in **Directory** → **Organizational units** and re-run.

T2 — Settings baseline applied, zero drift (Gate G1)

- **Procedure:**

```
python config_drift_detector.py --baseline ou-policy-baseline.json --scope Corp/Standard-Users/Pilot
```

- **Expected PASS:** the tool reports **0 drift** — every governance-critical setting on the pilot OU

matches `Baseline – OU Settings – Design` (external sharing, 2SV enforcement, per-service on/off, etc.).

- **FAIL:** one or more settings differ (the tool prints `DRIFT: <setting> expected=<x> actual=<y>`).

→ Either correct the live setting to match the baseline, or, if the live value is intentionally correct, update `ou-policy-baseline.json` under change control. Re-run until 0 drift.

T3 — Inheritance behaves as designed

- **Procedure:** Change a setting at a parent OU (in a change window) and confirm a child OU marked

"inherit" follows it, while a child marked "override" does not. Re-run the drift detector on both.

- **Expected PASS:** inherited children reflect the parent change; overridden children retain their

baseline value; drift detector remains 0 for both against the updated baseline.

- **FAIL:** an inherit child did not follow, or an override child changed unexpectedly. → Recheck the

child OU's inherit/override state in the console; reconcile the baseline.

T4 — Least-privilege delegation (Gate G2)

- **Procedure:** Run `python admin_role_auditor.py --output admin-role-audit-test.csv`. Also, signed

in **as the pilot help-desk member**, attempt an in-scope action (reset a pilot user's password) and an out-of-scope action (change a security policy).

- **Expected PASS:** the auditor shows the pilot help-desk member holding only `Admin Role – Help Desk

– Corp` and **no super-admin**; the super-admin count matches the known small number. The in-scope action succeeds; the out-of-scope action is **denied**.

- **FAIL:** the member has extra privileges/super-admin, or the out-of-scope action succeeds. → Tighten the custom role's privileges/scope in **Account** → **Admin roles** and re-run.

T5 — Alerts fire on test events (Gate G3)

- **Procedure:** For each `Alert – <condition> – Enforce`, trigger a controlled test event:
- `Alert – External-Sharing-Spike – Enforce` → externally share a pilot test document.
- `Alert – Admin-Privilege-Change – Enforce` → assign then remove a test role.
- `Alert – Suspicious-Login – Enforce` → follow Google's documented test guidance where available

(**verify in your tenant**; some alerts cannot be safely simulated — confirm via a real prior event in logs instead).

- **Expected PASS:** each alert appears in **Security** → **Alert center** and notifies the security-ops group within its expected latency.
- **FAIL:** an alert did not appear or did not notify. → Recheck the alert rule and recipients; confirm the alert type is enabled for your edition.

T6 — Health-check report generates from real data (Gate G3)

- **Procedure:**

```
python health_check_report.py --days 7 --output health-check-test.html
```

- **Expected PASS:** the report renders with real Reports API data — login summary, admin-activity summary, external-sharing summary, and a top-risk section — for the last 7 days, with no API errors.
- **FAIL:** the tool errors (auth/scope/`403`) or the report is empty. → Confirm the R4 credential has the read-only Reports scope and the Reports API is enabled; see troubleshooting.

T7 — Runbooks executed by delegated role (Gate G4)

- **Procedure:** Using the pilot help-desk role (not super-admin), execute:
- `Runbook – User Onboarding – Production` on a new pilot test user.
- `Runbook – User Offboarding – Production` on a departing pilot test user — first as a dry run:

```
python ops_config_helper.py offboard --user pilot.leaver@example.com --dry-run
```

then perform the documented console/API steps.

- `Runbook – Access Request Handling – Production` for a Shared Drive access request.
- one incident runbook (e.g. `Runbook – Incident: Data Exposure / Oversharing – Production`).
- **Expected PASS:** each runbook completes with its verification step green, performed **without super-admin**, and each touched artifact matches its runbook-named convention.

• **FAIL:** a step requires super-admin, the verification fails, or the runbook is ambiguous. → Fix the role scope or the runbook and re-run.

T8 — Drift detection catches an unplanned change

• **Procedure:** Deliberately change one governed setting on the pilot OU **out of baseline** (e.g. turn on external link sharing). Run `config_drift_detector.py` for the pilot scope.

• **Expected PASS:** the detector reports exactly that one drift with `DRIFT: <setting> expected=<baseline> actual=<changed>`.

• **FAIL:** the change is not detected. → Confirm the setting is covered by the baseline schema; extend the baseline/tool coverage. Revert the test change afterward.

Validation summary

Gate	Test(s)	Pass condition
G1 — OU/policy applied	T1, T2, T3	OU tree matches design; 0 drift vs. baseline; inheritance correct
G2 — Least-privilege	T4	Delegated role only; no unexpected super-admin; out-of-scope denied
G3 — Monitoring live	T5, T6	Every alert fired on its test event; health report renders from real data
G4 — Runbooks exercised	T7, T8	Runbooks completed without super-admin; drift detector catches unplanned change

All four gates must pass on the pilot before Expand. Re-run T2, T4, and T6 after each expansion batch.

Troubleshooting

Common failure modes when standing up and running the Google Workspace day-2 operations practice, and how to resolve them. Symptoms are grouped by area. Admin console paths reflect Google's public documentation at authoring time — where a label or capability may differ by edition or rollout, the guidance says **verify in your tenant.**

Authentication and API access (the Python tools)

Symptom: `403` **/** `insufficientPermissions` **or** `access_denied` **from any tool.**

• The credential lacks the required **read-only scope**, or the scope was not authorized for the service account under domain-wide delegation.

- Fix: in **Admin console** → **Security** → **API controls** → **Domain-wide delegation**, confirm the client ID

is listed with **exactly** the read-only scopes each tool names in its header (Directory readonly, Reports audit readonly, etc.). Re-authorize if a scope is missing. For OAuth-user credentials, re-consent with the required scopes.

Symptom: `403 admin.googleapis.com ... has not been used in project ... or it is disabled`**.**

- The **Admin SDK API** (and/or the Reports API) is not enabled in the Cloud project backing the

credential.

- Fix: enable the **Admin SDK API** in the associated Google Cloud project (R4), wait a few minutes,

and retry.

Symptom: a tool prints `no live credential ... rendering SAMPLE/offline`**.**

- No Application Default Credentials are available, or the client libraries are not installed.
- Fix: `pip install google-api-python-client google-auth` and provide a credential (ADC via

`gcloud auth application-default login`, or a service-account credential path via the standard `GOOGLE_APPLICATION_CREDENTIALS` environment variable). The sample/offline output is for inspection only — it is clearly labelled and must not be treated as real tenant data.

Symptom: Reports API returns empty results for a window you know had activity.

- Reports data has an **ingestion delay** (events can take time to appear) and a per-service

retention window; a query outside retention returns nothing.

- Fix: widen `--days`, allow for ingestion latency, and confirm the event's service retention window

for your edition (**verify in your tenant**). For long-term needs, export logs (e.g. BigQuery).

OU tree and settings

Symptom: a setting you changed on a child OU did not take effect / reverted.

- The child OU is set to **Inherit** from its parent, so parent values win, or another admin changed

it back under a different session.

- Fix: in the service's settings, confirm the child OU shows **Override** (not Inherit) for that

setting. Re-run `config_drift_detector.py` for that OU to confirm.

Symptom: a policy is not applying to the users you expect.

- Users may not be in the OU you think, or the policy is targeted by a **configuration group** rather

than the OU (or vice versa).

- Fix: run `ou_policy_assessment.py` and check `users_per_ou`; confirm each user's actual `orgUnitPath`

in **Directory** → **Users**. If the policy is group-targeted, confirm the group membership — and **verify group-based policy targeting is available for that service in your tenant.**

Symptom: `config_drift_detector.py` **reports** `VERIFY: <setting> ... actual=verify-in-console`**.**

- That setting has no wired read API in the reference tool for your edition; the tool refuses to guess

a value.

- Fix: check the setting manually in the console, or extend `read_live_settings()` with the concrete

Admin SDK call your edition supports (verify the endpoint/scope names). This is expected behavior, not an error.

Symptom: drift is reported but the live value is actually the intended one.

- The **baseline** is stale, not the tenant.
- Fix: update `ou-policy-baseline.json` under change control to the correct intended value, then

re-run. Never edit the baseline silently — it is the governed source of truth (treat as code).

Admin roles and delegation

Symptom: `admin_role_auditor.py` **exits** `4` **(super-admin count exceeds threshold).**

- More accounts hold super-admin than the break-glass minimum.
- Fix: in **Account** → **Admin roles** → **Super Admin**, review each assignment; move routine operators to

scoped custom roles and keep only break-glass accounts as super-admin. Re-run the auditor.

Symptom: a delegated help-desk admin cannot perform an in-scope action.

- The custom role is missing the specific privilege, or its **OU scope** does not include the target

user.

- Fix: in **Account** → **Admin roles**, add the missing privilege to the custom role and confirm the OU

scope covers the target population. Note that some privileges are **not OU-scopable** — **verify in your tenant** which privileges support OU scoping.

Symptom: an out-of-scope action unexpectedly succeeded during testing (T4).

- The role grants more than intended, or the operator also holds a broader role/super-admin.
- Fix: run the auditor CSV, find every role the operator holds, and remove the excess.

Monitoring and alerts

Symptom: a configured alert did not fire on its test event (T5).

- The alert type may not be enabled for your edition, the rule condition/threshold was not met, or the

event has not yet ingested.

• Fix: in **Security** → **Alert center** → **Settings**, confirm the alert is enabled and its recipients are correct; allow for ingestion latency; **verify the alert type is available for your edition.** Some alerts cannot be safely simulated — validate against a real prior event in the logs instead.

Symptom: alert notifications are not reaching the security-ops team.

• Recipients are set to an individual on PTO, or a group address that is not receiving external/system mail.

• Fix: set recipients to the security-ops **group** and confirm the group can receive the notifications; send a test.

Runbooks and lifecycle helper

Symptom: `ops_config_helper.py --apply` **prints that live writes are "not wired".**

• By design, the reference helper does not perform live writes, to prevent accidental changes.

• Fix: implement each step against the Admin SDK for your environment, **keeping the `--apply` + typed-confirmation gate**. Until then, run in dry-run and perform the documented console steps manually.

Symptom: offboarding left data inaccessible.

• Drive/data ownership transfer was skipped or the account was deleted before transfer.

• Fix: follow `Runbook – User Offboarding – Production` in order — **transfer ownership (Data Transfer) before** suspending/deleting; if under legal hold, retain rather than delete. Dry-run first to see the full ordered checklist.

Compliance boundary reminder

If a question is whether a control **satisfies** CMMC/NIST 800-171/ITAR/DFARS or an assessor's determination — that is **out of scope for troubleshooting here.** The operations practice supplies the technical configuration, monitoring, and evidence; the determination belongs to your compliance, legal, and assessment (C3PAO) partners.

Best Practices

Durable practices for running Google Workspace day-2 operations. These outlast any single change and should govern how the operations team works from go-live onward.

OU tree and policy governance

• **Keep the OU tree shallow and purpose-driven.** Model *policy differences*, not the HR org chart.

Every OU should exist because a population needs a different setting; if two OUs have identical policy, merge them.

• **Prefer inheritance; override deliberately.** Set the safe default at a parent and inherit down. Every override is a maintenance liability — document *why* each override exists in the baseline.

• **Treat the settings baseline as code.** `ou-policy-baseline.json` is the source of truth for intended configuration. Change it only through change control, keep it in version control, and let the drift detector enforce that live == baseline.

• **Use configuration groups only where policy must follow membership,** not OU placement — and **verify group-based policy targeting for that service in your tenant** before relying on it.

Least-privilege administration

• **Super-admin is break-glass, not a job title.** Keep the count minimal, use dedicated break-glass accounts with strong 2SV, and never do routine work as super-admin.

• **Scope custom roles to OUs** wherever the privilege supports it, so help desk touches only its population.

• **Re-audit roles on a schedule.** Run the admin-role auditor monthly; every new super-admin or root-scoped role should be a deliberate, logged decision.

• **Offboard admin rights first.** When an operator leaves, remove role assignments before anything else, and confirm with the auditor.

Monitoring, alerting, and health

• **Monitor the security-relevant few, not everything.** Suspicious logins, admin-privilege changes, external-sharing spikes, and OAuth grants catch most real problems; alert fatigue kills the rest.

• **Route alerts to a group, not a person,** so coverage survives PTO and turnover.

• **Run the health check on a cadence and keep the output.** A dated health report is both an early-warning signal and audit evidence.

• **Mind log retention.** Know your edition's audit-log retention windows; if compliance needs longer, export logs (e.g. to BigQuery) — **verify the current mechanism and windows for your edition.**

Runbooks and change control

• **Every routine task is a named runbook.** If it is done more than twice, it gets a `Runbook – <task> – Production`. Runbooks are the antidote to key-person risk.

• **Dry-run destructive operations by default.** Offboarding, device wipes, and bulk changes should print what they *would* do before they do it — the tooling defaults to `--dry-run` for this reason.

• **Change control for anything that touches mail flow, sharing, identity, or security policy.** These are high-impact; require approval, a change window, and a documented rollback.

• **Exercise incident runbooks before you need them.** Run a tabletop for account compromise and data

exposure so the first real incident is not the first rehearsal.

Tooling and credentials

• **Read-only by default.** Assessment, validation, health, and drift tooling should never need write access. Grant the Python tools **only** read-only Directory/Reports/Groups scopes.

• **No secrets in code, ever.** Use OAuth / ADC / a secured service-account credential supplied at runtime. Never commit keys; never paste credentials into a runbook.

• **Version the baseline and the reports.** Store the baseline, drift reports, role audits, and health reports in an access-controlled, durable evidence store (R6).

Compliance posture

• **Provide evidence, defer determinations.** The operations practice produces the technical control configuration, monitoring, and evidence. Whether that satisfies CMMC/NIST 800-171/ITAR/DFARS is a determination for your compliance, legal, and assessor (C3PAO) partners — do not assert certification from the technical side.

Deliverables Summary

Everything included in the **Google Workspace Day-2 Administration & Operations Runbook**, on one page.

Documentation

• **Solution Overview** — the operating model plus the OU/policy-governance architecture (OU tree, policy inheritance, settings baseline, change control) and the Assess → Design → Configure → Pilot → Validate → Expand deployment model.

• **Requirements Guide** — the solution-specific prerequisites (steady-state go-live, super-admin + admin-team structure, Reports API access/retention, a read-only control-plane identity for the tooling, a target-state design/baseline, and change-control/evidence), each with how to verify and how to implement if missing — layered on the shared Google-category requirements.

• **Implementation Guide** — exact, ordered steps to design and build the OU tree and settings baseline, establish least-privilege admin-role delegation, stand up monitoring/alerting and health checks, and author the runbook library — pilot-first, every artifact named, with pass/fail gates and rollbacks.

• **Testing & Validation** — eight validation procedures (T1–T8) mapped to the four gates (OU/policy applied, least-privilege, monitoring live, runbooks exercised), each with an expected pass/fail signal.

• **Troubleshooting** — common failure modes and resolutions (API scope/auth `403`s, drift false

positives, inheritance confusion, alert-simulation limits, retention gaps).

• **Best Practices** — durable practices for OU/policy governance, least-privilege administration, monitoring, runbooks/change control, tooling/credentials, and compliance posture.

• **Runbook Library table of contents** — the named runbooks (onboarding, offboarding, group/Shared Drive lifecycle, device action, request handling, three incident runbooks, change control, drift review).

The 5 Python tools

All tools are **read-only or dry-run by default**, contain **no credentials/keys**, authenticate via OAuth/ADC or a domain-wide-delegated service account with **read-only** Admin SDK scopes, and expose `--help`. They require `pip install google-api-python-client google-auth` for live API calls and fall back to a documented **offline/sample mode** so they can be inspected without a tenant.

1. `ou_policy_assessment.py` — *Assessment / discovery (read-only).* Enumerates the OU tree (Directory `orgunits.list`), users/admins per OU, groups, and an admin-role-assignment summary, and writes a JSON baseline snapshot. This is the Assess-stage artifact and the seed for the intended settings baseline.

1. `config_drift_detector.py` — *Validation / drift detection (read-only).* Compares the live configuration for a given OU scope against a curated **baseline JSON** (`ou-policy-baseline.json`) and reports each `DRIFT: <setting> expected=<x> actual=<y>`, exiting non-zero when drift exists so it can gate CI/scheduled checks.

1. `admin_role_auditor.py` — *Delegation / control check (read-only).* Lists every admin-role assignment (`roleAssignments.list` + `roles.list`), flags **super-admins** and root-scoped roles, and exports a CSV audit for least-privilege review (Gate G2).

1. `health_check_report.py` — *Health / monitoring report generator (read-only, Reports API).* Pulls recent activity from the Admin SDK **Reports API** (login, admin, drive/token events) for the last N days and renders an HTML health report — login summary, admin-activity summary, external-sharing summary, and a top-risk section — for the daily/weekly health cadence (Gate G3).

1. `ops_config_helper.py` — *Operations / config helper (dry-run default, `--apply` gated).* Standardizes routine runbook inputs (e.g. offboarding checklist, group/Shared-Drive lifecycle scaffolding) and, by default, **prints the exact API calls it would make** without changing anything; write actions require an explicit `--apply` and confirmation. Backs the lifecycle runbooks with a safe, repeatable helper.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

admin_role_auditor.py

config_drift_detector.py

health_check_report.py

ops_config_helper.py

ou_policy_assessment.py